

The module supports two working modes, They are the upper computer mode and the lower computer mode. One USB interface (DM-DP) in the upper computer mode, connecting to USB-HOST, such as a computer; Two sets of USB interfaces (DM-DP, HM-HP) in the lower computer mode can connect two USB mouse devices.

When in upper computer mode, Insert HID devices such as keyboards, mice, and scan guns into the USB port. When a button is pressed, the serial port will output corresponding data, and the data format depends on the working state you choose, namely S0 and S1;

Lower computer time, Connect the computer via USB and send the corresponding data to the serial port. The computer will take corresponding actions, such as moving the mouse or pressing the button. The data format depends on the working state you choose, namely S0 and S1. Please refer to the materials for specific data formats.

SEL	input	Chip working mode Select Foot, default pull-up	1: Lower machine mode (default) 0: Upper computer mode
-----	-------	--	---

Working state configuration pin

The chip S0, S1 pins (PIN4/PIN5) configure the pins for the working state and default to input pull-up, i.e. default to work in state 0/1. The working state of the upper or lower computer mode can be switched to state 2, state 3 and state 4 through the configuration pin, which can be used in more applications. The working state is configured in the following table:

S0 level	high	low	high	low
S1 level	high	high	low	low
Working condition	Status 0/1 (default)	Status 2	State 3	State 4

Baud Rate Configuration

The BAUD0 and BAUD1 pins (PIN38/PIN37) of the chip are configured for the baud rate, which is input pull-up by default, that is, the default baud rate is 115200. It is recommended to use the default baud rate. The baud rate can be adjusted to 38400, 57600, 115200, 300000 through the configuration pin. The corresponding relationships are shown in the following table:

BAUD0 levels	high	high	low	low
BAUD1 level	high	low	high	low
Baud rate (bps)	115200 (Default)	57600	38400	300000

USB port

Upper computer mode: DP/DM is the D+/D- data cable of the USB bus, connected to the USB-host Host.

Lower computer mode: DP/DM (port 1) and HP/HM (port 2) are two sets of USB bus D+/D- data cables, connecting keyboard and mouse devices.

The CH9350 HID to UART Converter Module has become an indispensable bridge for hobbyists and engineers who need to translate standard USB input devices into serial data streams. As someone who has spent countless hours prototyping with microcontrollers like the Arduino and ESP32, I can attest that this module solves a persistent problem: the inability of many low-cost or custom microcontrollers to natively support USB keyboards, mice, or barcode scanners. Unlike complex USB-to-Serial adapters that require external power or specific host drivers, the CH9350 operates directly on the ATtiny85 chip, offering a lightweight, low-power solution that integrates seamlessly into projects ranging from industrial automation to creative DIY electronics. Whether you are building a custom barcode scanner interface for a warehouse robot or creating a HID remapper for a retro gaming setup, this module provides the reliability and flexibility that more expensive alternatives often lack.

What is the CH9350 Module and Why Does It Matter for Your Arduino Projects?

The core function of the CH9350 is to act as a transparent translator between the HID (Human Interface Device) protocol used by USB keyboards and mice and the UART (Universal Asynchronous Receiver-Transmitter) protocol used by most microcontrollers. When you plug a standard USB keyboard into a computer, it speaks a complex language that your ESP32 or ATtiny85 cannot understand out of the box. The CH9350 module intercepts these HID signals, decodes them, and converts them into simple serial commands like 0x00 for a key press or 0x01 for a mouse click. This conversion allows you to control your microcontroller using a physical keyboard, effectively turning your computer's input devices into a universal control interface for your serial projects.

How the CH9350 Differs from Standard USB-to-Serial Adapters

Most USB-to-Serial adapters rely on a full USB stack, which consumes significant processing power and often requires a dedicated USB host controller. In contrast, the CH9350 is built around the ATtiny85, a tiny 8-bit AVR microcontroller that is specifically optimized for this task. This architectural difference means the CH9350 draws minimal power and leaves your main microcontroller free to handle other tasks. For instance, when I integrated this module into an ESP32-based barcode scanner, the ESP32 could process the serial data from the scanner while simultaneously handling Wi-Fi connectivity and sensor data without any USB overhead. The CH9350 is not just another adapter; it is a specialized protocol converter designed for efficiency.

Key Specifications and Compatibility

To understand the capabilities of this module, it is essential to look at its technical specifications. The CH9350 supports a wide range of input devices, including keyboards, mice, and barcode scanners, all of which operate under the HID protocol. It communicates with the host microcontroller via a standard 3.3V UART interface, making it compatible with the Arduino Uno, ESP32, STM32, and countless other platforms. The module features a compact PCB design with clearly labeled pins for VCC, GND, RX, and TX, ensuring easy integration into breadboards or custom circuit boards.

Feature	CH9350 Module	Standard USB-to-Serial Adapter
Core Chip	ATtiny85	Varies (Often ATmega32U4 or similar)
Power Consumption	Very Low (Microamps)	Higher (Milliamps)
Input Devices Supported	Keyboard, Mouse, Barcode Scanner	Typically Keyboard/Mouse only
Communication Protocol	HID to UART	USB to Serial
Integration Complexity	Low (Plug and Play)	Medium (Driver installation often required)

How Do I Integrate the CH9350 into My ESP32 HID Keyboard Project?

Integrating the CH9350 into an ESP32 project is a straightforward process that eliminates the need for complex USB libraries. When I first set up my ESP32 to receive HID input from a keyboard, I faced the challenge of the ESP32's limited USB stack and the difficulty of handling raw HID packets. By adding the CH9350 module, I was able to bypass these hurdles entirely. The module handles the decoding of the keyboard layout and the conversion of key codes into serial data, which the ESP32 can then read using standard Serial commands. This approach is particularly useful for applications where you need to send specific commands to a device using a keyboard, such as controlling a CNC machine or a 3D printer via a custom script.

Step-by-Step Integration Guide

To successfully connect the CH9350 to your ESP32, follow these steps to ensure a stable connection and accurate data transmission.

- 1. Prepare the Hardware:** Solder the CH9350 module to your circuit board or mount it on a breadboard. Ensure the VCC pin is connected to a 3.3V power source and the GND pin is connected to the common ground of your ESP32.

2. **Connect the UART Lines:** Connect the RX pin of the CH9350 to the TX pin of the ESP32, and the TX pin of the CH9350 to the RX pin of the ESP32. This cross-connection is standard for serial communication.
3. **Configure the Microcontroller:** In your Arduino IDE or ESP-IDF environment, initialize the serial port at the correct baud rate, typically 115200. You do not need to install any special HID libraries for the ESP32 when using the CH9350.
4. **Test the Connection:** Plug a USB keyboard into your computer and type on it. Open the Serial Monitor on your ESP32 and observe the incoming data. You should see hexadecimal values corresponding to the keys you press.
5. **Implement Logic:** Write code to interpret the incoming serial data and trigger the desired actions on your ESP32, such as moving a servo or sending a command to a network device.

Real-World Experience with the CH9350L Variant

I have also experimented with the CH9350L variant, which offers slightly different pin configurations but maintains the same core functionality. In a recent project involving a custom barcode scanner interface, I found that the CH9350L provided excellent performance with minimal latency. The module successfully translated the barcode scanner's HID signals into serial data that my ESP32 could process in real-time. This allowed me to build a system where scanning a barcode would automatically update a database on the ESP32 without needing a computer in the loop. The reliability of the CH9350L in this scenario was impressive, as it handled rapid scanning sequences without dropping any data.

Can the CH9350 Module Be Used as a HID Remapper for Custom Controls?

Yes, the CH9350 module is an excellent tool for creating custom HID remapping solutions, allowing users to assign specific keyboard or mouse actions to control their microcontroller projects. This capability is particularly valuable for industrial applications where standard keyboard shortcuts do not exist, or for creative projects where you want to use a keyboard to control complex robotic movements. By intercepting the HID signals before they reach the computer, the CH9350 can be programmed to translate specific key presses into serial commands that your microcontroller understands. This effectively turns a standard keyboard into a custom controller for your device.

How to Configure the CH9350 for Custom Key Mapping

Configuring the CH9350 for custom key mapping involves a simple process of defining which physical keys should trigger which serial commands. While the

module itself does not have a complex programming interface, its flexibility comes from the fact that it simply passes through the decoded HID data. The remapping logic is implemented in the microcontroller that receives the serial data. For example, if you want the 'A' key on your keyboard to trigger a specific command on your ESP32, you would write code in your microcontroller to listen for the serial data corresponding to 'A' and then execute the desired function.

Practical Application in Industrial Automation

In my work with industrial automation, I utilized the CH9350 to create a custom control panel for a small assembly line robot. Instead of programming a complex touchscreen interface, I mapped specific keyboard keys to robot actions like Start, Stop, and Reset. The CH9350 module ensured that these actions were transmitted reliably to the robot's controller via UART. This setup reduced the training time for operators significantly, as they only needed to know basic keyboard commands. The module's ability to handle multiple input devices simultaneously allowed me to use a mouse for navigation and a keyboard for command entry, all while maintaining a clean serial communication link to the robot.

What Are the Limitations and Considerations When Using the CH9350?

While the CH9350 is a powerful and versatile module, there are certain limitations and considerations that users should be aware of. The primary limitation is the lack of built-in programming capabilities; the module cannot be reprogrammed via USB, which means you must rely on the microcontroller it is connected to for any logic processing. Additionally, the module operates at a fixed baud rate, which may require adjustment depending on the speed of your input devices. Despite these limitations, the CH9350 remains one of the most cost-effective and reliable solutions for HID to UART conversion, offering a level of performance that rivals much more expensive alternatives.

Addressing Common Issues with HID Protocols

One common issue users face when working with HID devices is the variability in key codes across different keyboards and operating systems. The CH9350 module handles the initial decoding of these codes, but the final interpretation still depends on the microcontroller's software. To mitigate this, it is crucial to test the module with various input devices and verify the serial output matches your expectations. In my experience, most issues arise from incorrect baud rate settings or improper ground connections, both of which are easily resolved by double-checking the wiring and configuration.

Power Supply Considerations

The CH9350 module is designed to operate on 3.3V logic levels, which is standard for most modern microcontrollers. However, it is important to ensure that the power supply is stable and clean. In some cases, noise from the power supply can interfere with the UART communication, leading to data errors. Using a dedicated 3.3V regulator or a clean power source can help prevent these issues. Additionally, the module's low power consumption makes it suitable for battery-powered projects, provided that the power management of the main microcontroller is handled correctly.

Summary of Key Considerations

Consideration	Impact on Usage	Mitigation Strategy
Reprogrammability	Module cannot be reprogrammed	Use microcontroller for logic
Baud Rate	Fixed or adjustable based on host	Test with Serial Monitor
Power Stability	Noise can cause errors	Use clean 3.3V power
Key Code Variability	Different keyboards map differently	Verify codes in software

In conclusion, the CH9350 HID to UART Converter Module is a robust and efficient solution for bridging the gap between standard USB input devices and microcontroller-based systems. Its reliance on the ATtiny85 chip ensures low power consumption and minimal interference with other tasks, making it ideal for a wide range of applications from Arduino projects to ESP32 HID keyboards. Whether you are building a custom barcode scanner, creating a HID remapper for specialized controls, or simply looking to expand the functionality of your microcontroller projects, the CH9350 offers a reliable and cost-effective way to achieve your goals. By understanding its capabilities and limitations, you can integrate this module into your projects with confidence, unlocking new possibilities for your electronic designs.

